

DAY_01

Date : 26.08.2026

Time : 5:30 P.M. to 8:00 P.M.

1. Write a program to calculate the total bill amount after applying a discount based on the purchase amount.
 2. Write a program to find the factorial of a number using a loop.
 3. Write a program to check whether a number is an Armstrong number.
 4. Write a program to generate the Fibonacci series up to N terms.
 5. Print different patterns of asterisk (*) using loops (e.g. triangle of *)
 6. Write a program to find principal diagonal sum of a matrix.
- =====

DAY_02

Date : 27.08.2026

Time : 5:30 P.M. to 8:00 P.M.

1. **Reverse a Number**
Write a program to reverse the digits of a given number using a loop.
Example: 12345 → 54321
2. **Check Palindrome Number**
Write a program to check whether a given number is a palindrome.
Example: 121 → Palindrome, 123 → Not Palindrome
3. **Check Prime Number**
Write a program to check whether a given number is prime or not.
4. **Print Prime Numbers in a Range**
Write a program to accept two numbers and print all prime numbers between them.
5. **Find Largest and Smallest Element in an Array**
Write a program to accept N integers into an array and find the largest and smallest elements without using built-in sorting methods.
6. **Calculate Sum and Average of Array Elements**
Write a program to accept N numbers into an array and calculate their total sum and average.

7. Count Even and Odd Numbers in an Array

Write a program to accept an array of integers and count how many elements are even and how many are odd.

8. Transpose of a Matrix

Write a program to accept a matrix and print its transpose.

Example: Rows become columns and columns become rows.

9. Find Sum of Each Row and Column in a Matrix

Write a program to accept a matrix and calculate the sum of every individual row and every individual column.

10. Count Frequency of Each Element in an Array

Write a program to accept N integers into an array and find how many times each element occurs.

Example: 2 3 2 5 3 2 → 2: 3 times, 3: 2 times, 5: 1 time

=====

DAY_03

Date : 29.08.2026

Time : 5:30 P.M. to 8:00 P.M.

1. Design a simple application (ex. Student Management System) using:
 - i. Classes and constructors
 - ii. Object references
 - iii. Association between classes
 - iv. Display students details
 2. Print Pascal's triangle.
 3. Print diamond star pattern.
 4. Print butterfly pattern.
- =====

DAY_04

Date : 31.08.2026

Time : 5:30 P.M. to 8:00 P.M.

1. Demonstrate boxing and unboxing
2. Demonstrate autoboxing and auto-unboxing
3. Use wrapper utility methods such as:
 - i. `parseInt()`
 - ii. `parseDouble()`
 - iii. `valueOf()`
 - iv. `toString()`
 - v. `compareTo()`
4. Create String objects using:
 - i. String literals
 - ii. `new String()`
5. Compare strings using:
 - i. `==`
 - ii. `equals()`
6. Observe the behavior of String Constant Pool.
7. Perform operations using:
 - i. `length()`
 - ii. `charAt()`
 - iii. `substring()`
 - iv. `indexOf()`
 - v. `contains()`
 - vi. `replace()`
 - vii. `split()`
 - viii. `toUpperCase()`
 - ix. `toLowerCase()`
 - x. `trim()`
8. Create programs demonstrating:
 - i. `append()`
 - ii. `insert()`
 - iii. `delete()`
 - iv. `replace()`
 - v. `reverse()`
9. Compare the behavior of String, StringBuffer, and StringBuilder.

=====

DAY_05

Date : 01.09.2026

Time : 5:30 P.M. to 8:00 P.M.

1. Design a Smart Device Management System where:
 - i. Device is an abstract class
 - ii. Interfaces such as Connectable, Rechargeable, and Upgradeable define device capabilities
 - iii. Different devices implement one or more interfaces
 - iv. Use default methods for common functionality
 - v. Study when to prefer an abstract class over an interface
 2. Write a program to check number is positive , negative or zero.
 3. Write a program to reverse given String.
 4. Write a program to find out duplicate character.
 5. Write a program to count special character in a String.
- =====

DAY_06

Date : 02.09.2026

Time : 5:30 P.M. to 8:00 P.M.

1. Enumeration – Employee Department

Create an enum named Department with the values HR, IT, FINANCE, SALES, and MARKETING.

Create an Employee class containing employee ID, name, and department. Write a Java program to:

- Create employee objects.
- Display their details.
- Use the enum to display the department.
- Use switch to display a suitable message for each department.

2. Regular Inner Class – Bank Account

Create a class BankAccount containing account number, account holder name, and balance.

Create a **regular inner class** named AccountDetails that displays the account information by accessing the private members of BankAccount.

Write a program to demonstrate:

- Creation of the outer class object.

- Creation of the inner class object.
- Accessing outer-class private data through the inner class.

3. Method-Local Inner Class – Student Grade

Create a class Student containing student name, roll number, and marks.

Define a method calculateGrade() inside the Student class. Within this method, create a **method-local inner class** named Grade that determines the student's grade based on the marks:

- 90–100 → A
- 75–89 → B
- 60–74 → C
- 40–59 → D
- Below 40 → F

Display the student's details and grade.

4. Anonymous Inner Class – Shape

Create an interface Shape containing a method calculateArea().

Write a Java program using an **anonymous inner class** to implement the Shape interface for calculating the area of a rectangle.

Accept the length and breadth from the user and display the calculated area.

5. Static Nested Class – Company and Employee

Create a class Company containing company name and location.

Create a **static nested class** named Employee containing employee ID and employee name.

Write a Java program to:

- Create an object of the static nested Employee class without creating an object of Company.
- Display employee details along with company information.
- Demonstrate the difference between accessing static and non-static members of the outer class.